

AN INTRO TO R, RSTUDIO & CAUSAL INFERENCE

Camila Pacheco Katrín Björnsdóttir Blandine Lyonnard

WELCOME!

SESSION GOALS

By the end of this session, you'll be able to:

-  Explain why ecological systems required multivariate approaches
-  Distinguish correlation from causation and identify potential confounding
-  Use R and RStudio to explore ecological data
-  Understand the basics of tidy data
-  Create simple plots using `ggplot2`

ECOLOGICAL AND BIOLOGICAL DATA CAN BE COMPLEX!

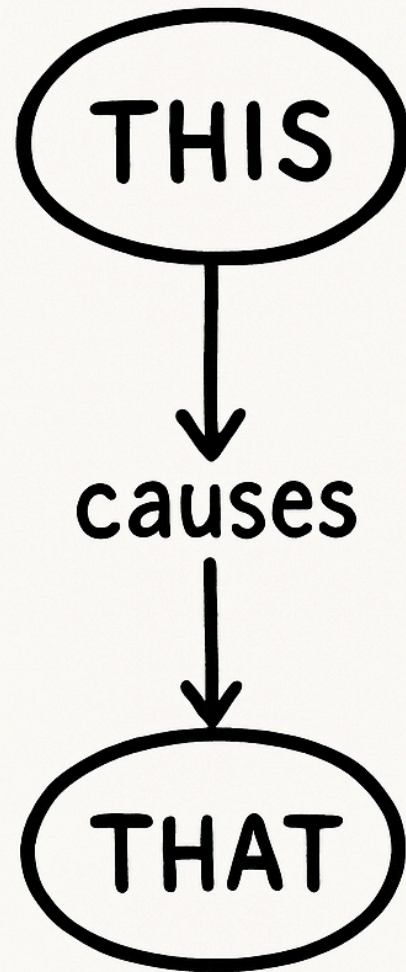
- Generalizing require numerical estimation of difference association.
- Hierarchical structure in the data
- Many covariates and grouping factors
- Unbalanced study/experimental design

BLUEBELLS



BLUEBELLS

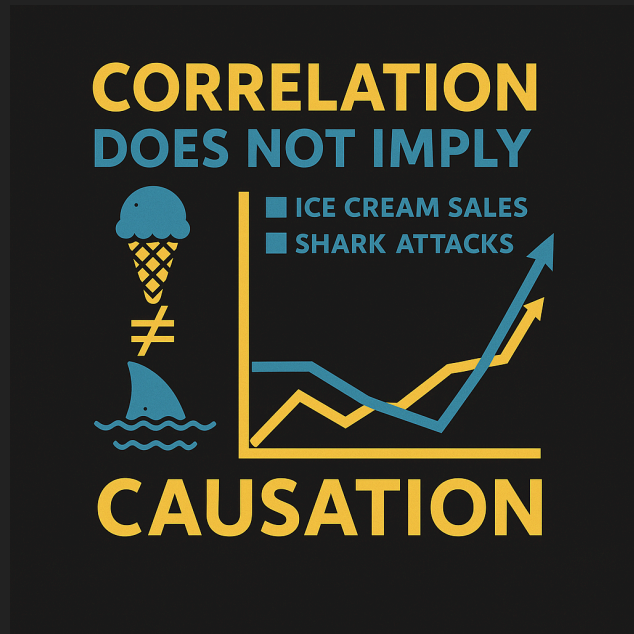




Causal
Inference

WHAT IS CAUSAL INFERENCE? 🤔

It's the process of determining cause-and-effect relationship.



- Moving beyond **correlation** to understand **causal relationships**
- Causal inference is about answering questions like: **"Does X cause Y?"**

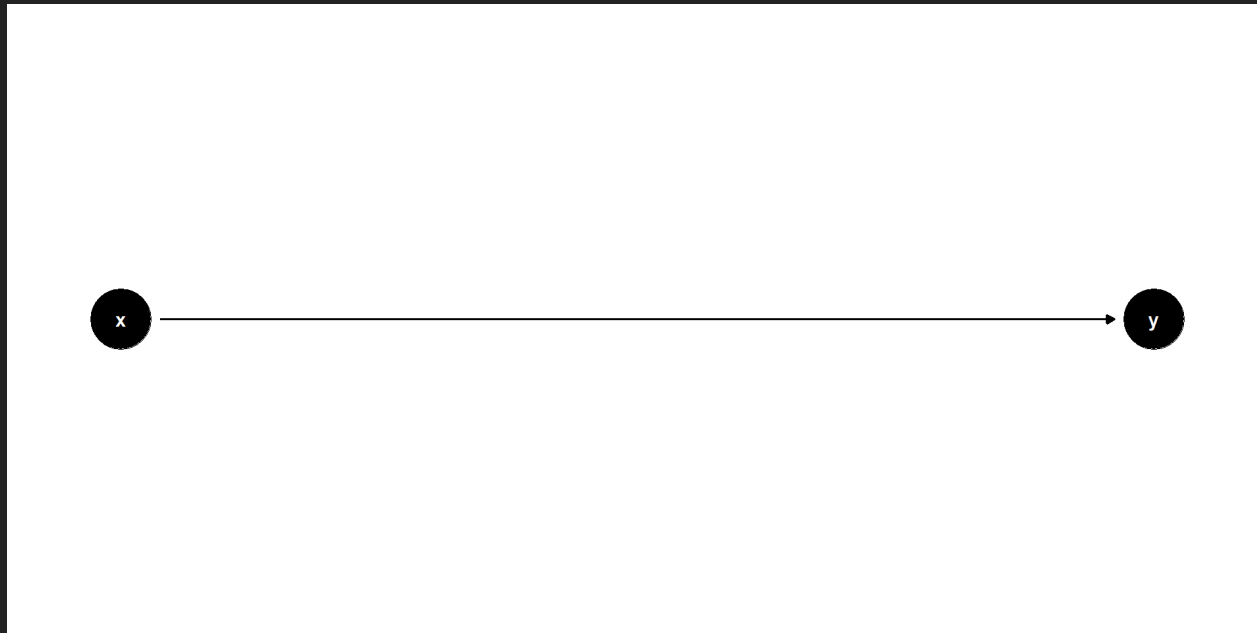
WHY IS THIS IMPORTANT IN ECOLOGY?

- Ecological systems are complex
- Many things can affect each other at the same time
- We need tools to figure out which relationships are **real causes**, not just coincidences

A SIMPLE TOOL: DAGS

A **DAG** (Directed Acyclic Graph) is a simple diagram to help us think clearly about cause and effect.

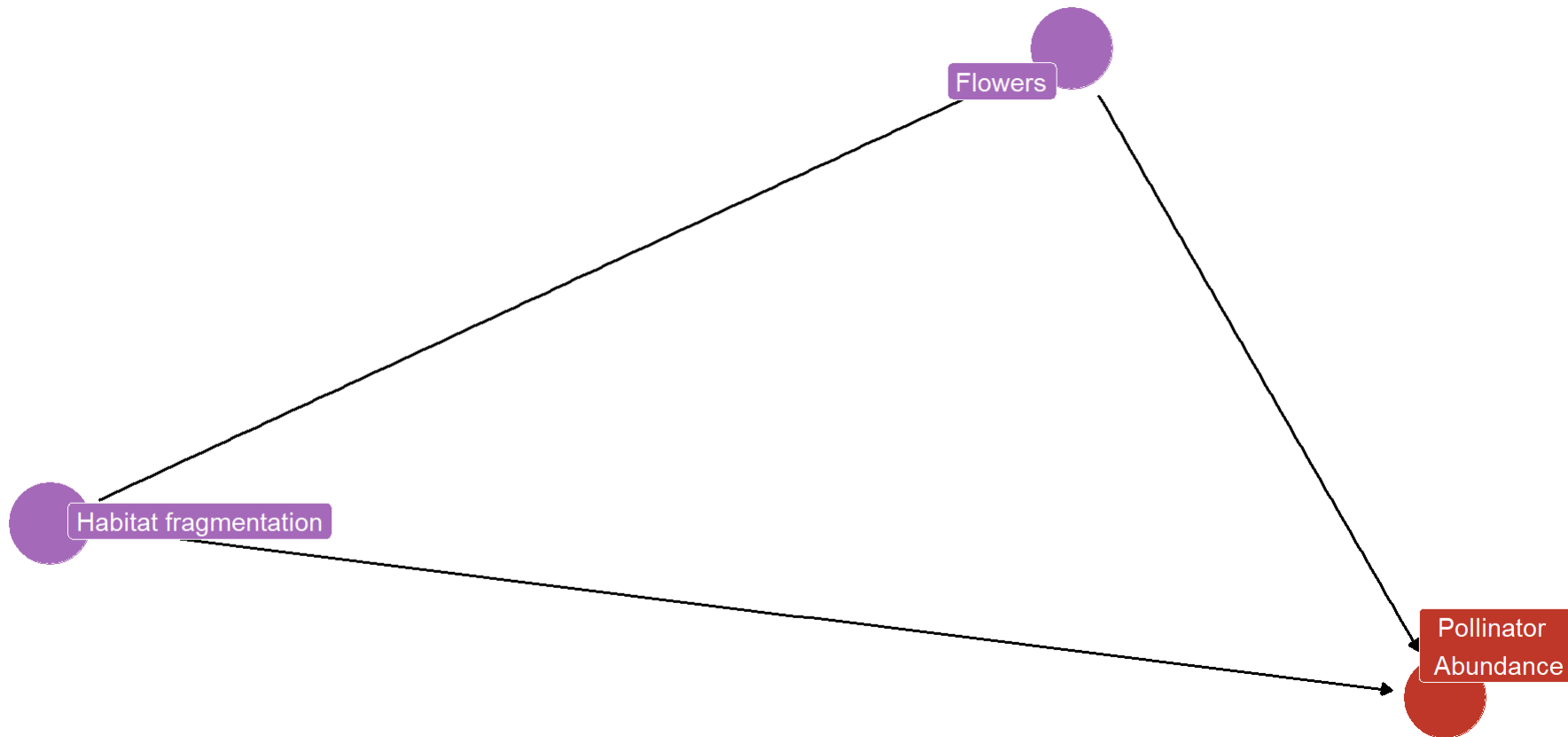
- Circles = variables (things we measure)
- Arrows = “causes”



DAG EXAMPLE

Fragmentation affects pollinators **directly** and **indirectly** (by reducing flowers).

DAG: Habitat → Flowers → Pollinators



HOW DAGS HELP

- They help us ask better research questions
- They show us what to measure and control for
- They help us avoid confusing correlation with causation

WHAT IS R AND RSTUDIO?



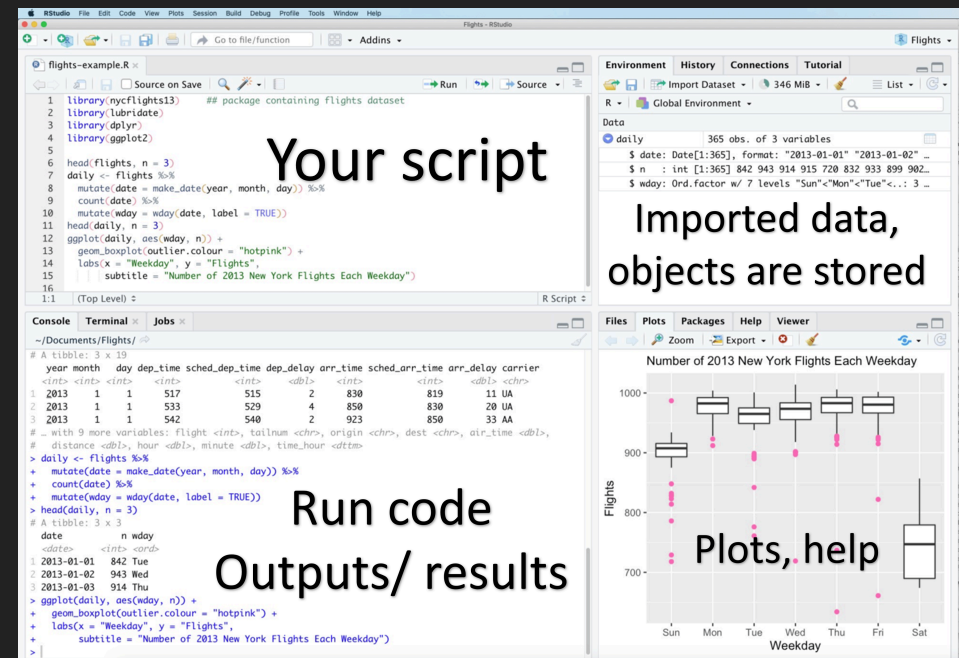
- Full-featured programming language for data science
- Especially powerful in ecological and environmental science 🐍🌍
- Remember: R is prone to errors!

- RStudio is a user-friendly interface (IDE) for R that makes your work more efficient.



Tip

Use **Ctrl + Enter** to run code in RStudio



Your script

Imported data,
objects are stored

Run code

Outputs/ results

Plots, help

R STUDIO PROJECTS

- Keeps your files organized in one place
- Stores scripts, data, outputs, and more
- To create a new project:
- Go to **File** → **New Project...** → **New Directory** (or **Existing Directory**)
- If using an existing folder: **New Project** → **Existing Directory**



Use projects to avoid broken file paths and messy folders!

COURSE MATERIAL



 **Important**

<https://github.com/Lacapary/BIO503>

CODING ETIQUETTE

USE COMMENTS TO ORGANIZE YOUR SCRIPT & FOR OTHERS!

```
1  #Library----  
2  
3  #Some code----  
4  
5  #Example----  
6  x <- 9 # Assigning the value to the variable x
```

R PACKAGE

- Package is a collection of R functions, data sets, and other resources bundled together for specific purposes.
- To install a package, type:
 - `install.packages("package-name")`
- **You only need to install packages once**
- Load the packages, type:
 - `library(package-name)`

EXAMPLE

```
1 install.packages("dplyr")  
2 library(dplyr)
```

INTRO TO tidyverse



PIPE OPERATOR

- One of the key features of tidyverse is the possibility to chain functions in an effective way using the pipe operator `%>%` or `|>`.
- Pipes pass the results from one function directly into the next function connected to each other
- **The pipe basically means “and then”.**

Note

the keyboard shortcut for `|>` is `Ctrl+Shift+M` (Windows & Linux) or `Cmd+Shift+M` (Mac).

PIPE OPERATOR

First, I'll grab the coffee grounds, then I'll fill up the coffee maker with water, hit the start button, wait for it to brew, and finally pour myself a cup

```
1  get("coffee_grounds", "23")    |>
2    fill_up("coffee_maker", "water")    |>
3    on(start_button)    |>
4    put(cup)
```

TIDY DATA CONCEPT

- Each variable forms a column
- Each observation forms a row
- Each type of observational unit forms a table

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174604898
China	1999	212258	1272915272
China	2000	213766	1280425583

variables

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174604898
China	1999	212258	1272915272
China	2000	213766	1280425583

observations

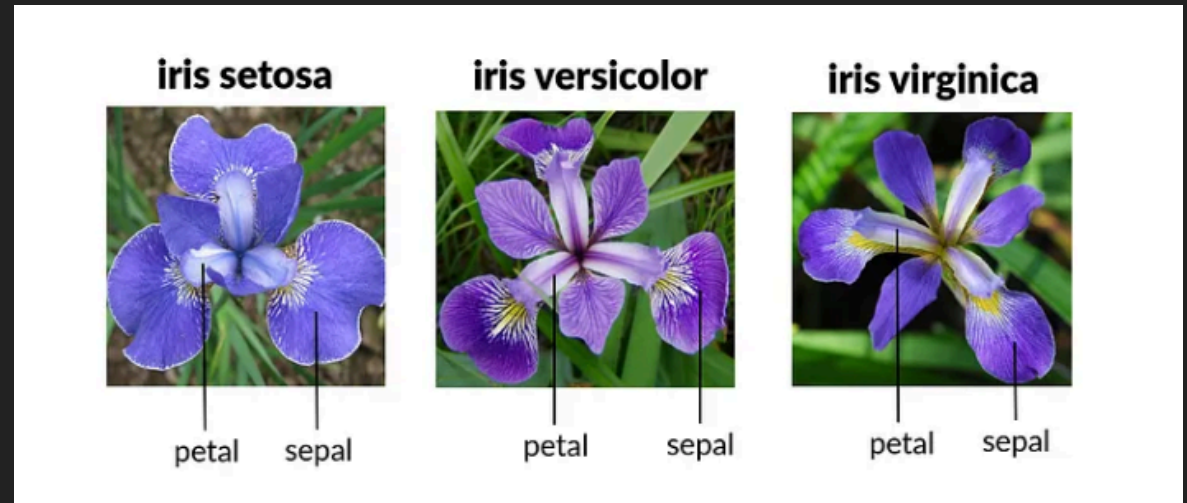
country	year	cases	population
Afghanistan	99	745	19987071
Afghanistan	00	2666	20595360
Brazil	99	37737	172006362
Brazil	00	80488	174604898
China	99	212258	1272915272
China	00	213766	1280425583

values

Source: R for Data Science. Golemund and Wickham.

USING THE **iris** DATASET

- The iris **flower dataset** was collected by Edgar Anderson, an American botanist, in the 1920s.
- It includes measurements of 150 flowers across 3 species: *setosa*, *versicolor*, and *virginica*.



IMPORTING DATA IN R

R can import data from files in many different formats.

- csv files with the **readr** package
- excel files with the **readxl** package
- xlm files with the **xlsx** package
- and more...

READING DATA

```
1 library(tidyverse)
2 iris <- read_csv("Data/iris.csv")
```

```
1 # R basic function
2 iris <- read.csv("Data/iris.csv")
```

EXPLORING THE DATA

Check that your data was imported without any mistakes

```
1 head(iris) # Displays the first rows of a df
```

```
# A tibble: 6 × 6
  ...1 Sepal.Length Sepal.Width Petal.Length Petal.Width Species
<dbl>   <dbl>         <dbl>         <dbl>         <dbl> <chr>
1     1         5.1         3.5         1.4     0.2 setosa
2     2         4.9         3         1.4     0.2 setosa
3     3         4.7         3.2         1.3     0.2 setosa
4     4         4.6         3.1         1.5     0.2 setosa
5     5         5         3.6         1.4     0.2 setosa
6     6         5.4         3.9         1.7     0.4 setosa
```

```
1 tail(iris) # Displays the last rows of a df
```

```
# A tibble: 6 × 6
  ...1 Sepal.Length Sepal.Width Petal.Length Petal.Width Species
<dbl>   <dbl>         <dbl>         <dbl>         <dbl> <chr>
1   145         6.7         3.3         5.7     2.5 virginica
2   146         6.7         3         5.2     2.3 virginica
3   147         6.3         2.5         5       1.9 virginica
4   148         6.5         3         5.2     2   virginica
5   149         6.2         3.4         5.4     2.3 virginica
6   150         5.9         3         5.1     1.8 virginica
```

EXPLORING THE DATA

```
1 summary(iris) # Gives you a summary of the data
```

```
...1      Sepal.Length      Sepal.Width      Petal.Length
Min.   : 1.00      Min.   :4.300      Min.   :2.000      Min.   :1.000
1st Qu.: 38.25      1st Qu.:5.100      1st Qu.:2.800      1st Qu.:1.600
Median : 75.50      Median :5.800      Median :3.000      Median :4.350
Mean   : 75.50      Mean   :5.843      Mean   :3.057      Mean   :3.758
3rd Qu.:112.75      3rd Qu.:6.400      3rd Qu.:3.300      3rd Qu.:5.100
Max.   :150.00      Max.   :7.900      Max.   :4.400      Max.   :6.900
Petal.Width      Species
Min.   :0.100      Length:150
1st Qu.:0.300      Class :character
Median :1.300      Mode  :character
Mean   :1.199
3rd Qu.:1.800
Max.   :2.500
```


MANIPULATING DATA



FROM WIDE TO LONG

The iris data are organized in “wide” format. Let’s transform it in “long” format

```
1 iris_long <- iris |> pivot_longer(  
2     cols = -Species,  
3     names_to = "trait",  
4     values_to = "measurement")  
5 head(iris_long)
```

```
# A tibble: 6 × 3  
  Species trait      measurement  
  <chr>   <chr>         <dbl>  
1 setosa ...1             1  
2 setosa Sepal.Length    5.1  
3 setosa Sepal.Width     3.5  
4 setosa Petal.Length    1.4  
5 setosa Petal.Width     0.2  
6 setosa ...1             2
```

GROUP_BY AND SUMMARIZE

```
1 iris_means <- iris |>
2   group_by(Species) |>
3   summarize(SL_mean = mean(Sepal.Length),
4             SL_se = sd(Sepal.Length) / sqrt(n()))
5
6 head(iris_means)
```

```
# A tibble: 3 × 3
```

	Species <chr>	SL_mean <dbl>	SL_se <dbl>
1	setosa	5.01	0.0498
2	versicolor	5.94	0.0730
3	virginica	6.59	0.0899

FILTER - SUBSET ROWS

```
1 iris_versicolor <- iris |>
2   filter(Species == "versicolor")
3
4 iris_pl <- iris |>
5   filter(Petal.Length > 2)
```

SELECT - SUBSET COLUMNS

```
1 iris_subset <- iris |>  
2   select(Species, Petal.Width, Petal.Length)
```

MUTATE

```
1 iris_log <- iris |>
2   mutate(log.Sepal.length = log(Sepal.Length))
3
4 head(iris_log)
```

```
# A tibble: 6 × 7
  ...1 Sepal.Length Sepal.Width Petal.Length Petal.Width Species
<dbl>   <dbl>         <dbl>         <dbl>         <dbl> <chr>
1     1     5.1         3.5         1.4         0.2 setosa
2     2     4.9         3         1.4         0.2 setosa
3     3     4.7         3.2         1.3         0.2 setosa
4     4     4.6         3.1         1.5         0.2 setosa
5     5     5         3.6         1.4         0.2 setosa
6     6     5.4         3.9         1.7         0.4 setosa
# i 1 more variable: log.Sepal.length <dbl>
```

PLOTTING DATA

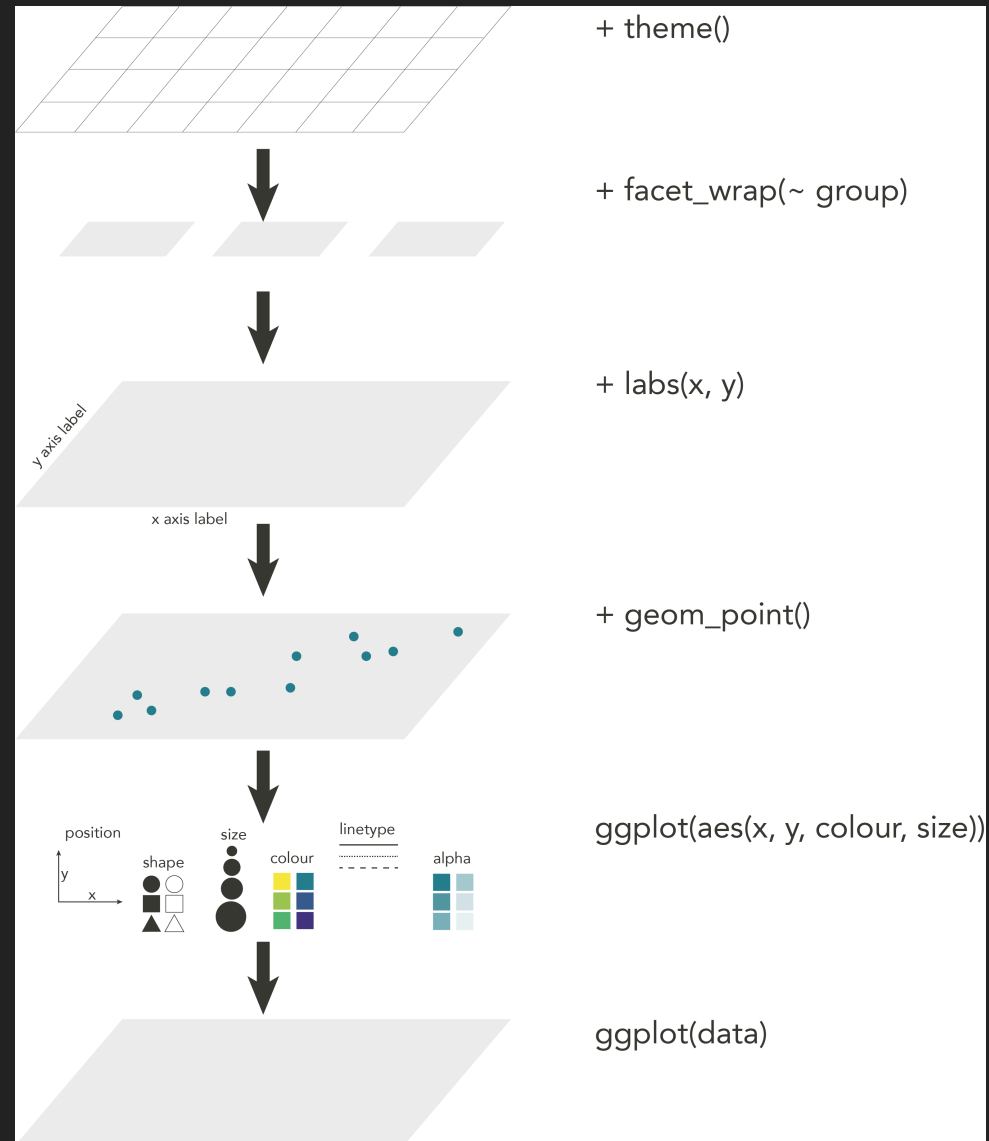
You can plot graphs using the `ggplot2` package (part of the tidyverse).



Note

`ggplot` functions are chained using a `+`

GGPLOT LAYERS



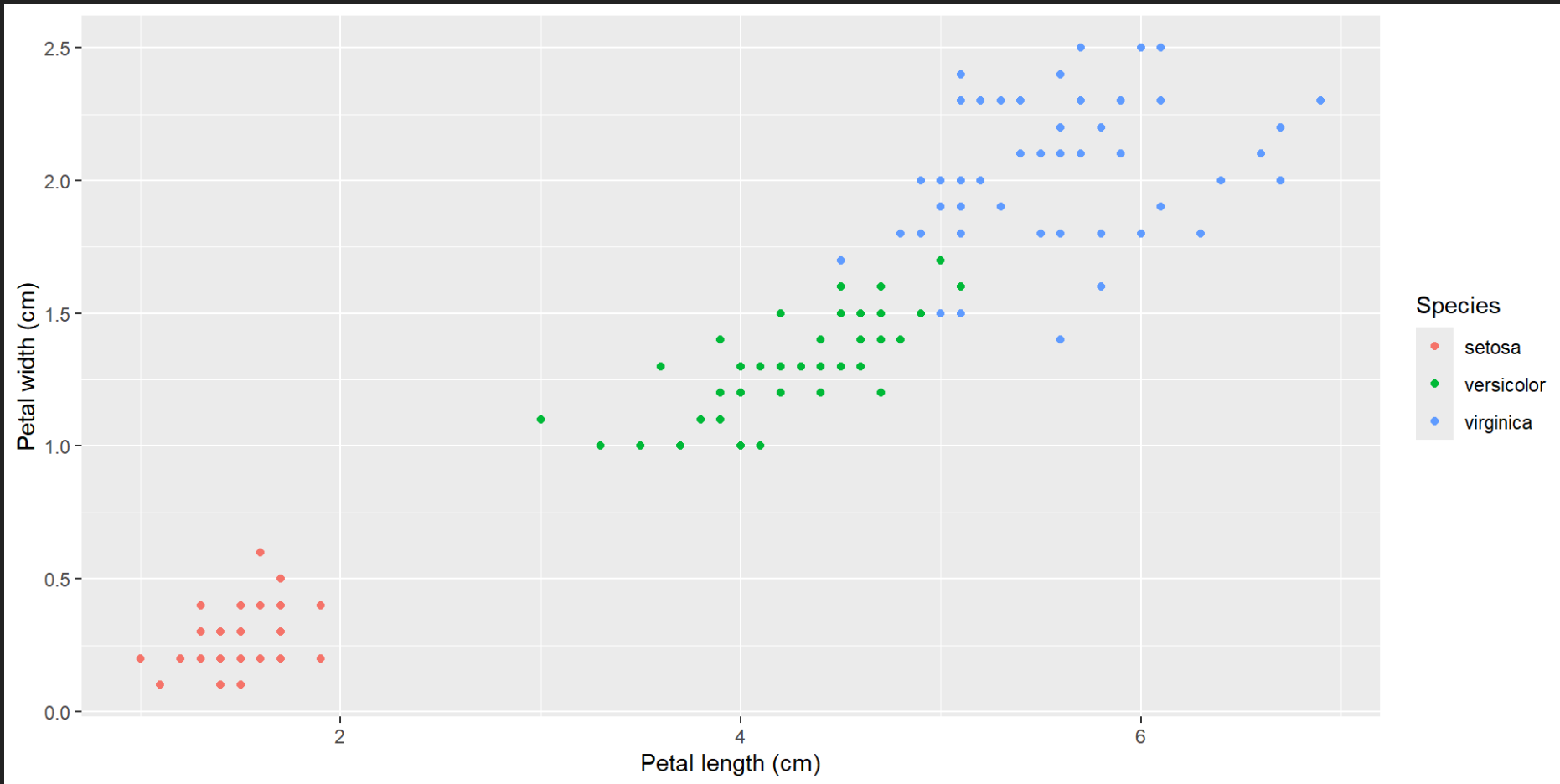
Source: <https://biostats-r.github.io/>

PLOTTING IRIS

Lets say we want to plot the relationship between petal length and petal width for each species.

```
1  p <- iris %>%
2    ggplot(aes(
3      x = Petal.Length,
4      y = Petal.Width,
5      color = Species # to assign a color to each group
6    )) +
7    geom_point() + # to plot a scatter plot
8    labs(
9      x = "Petal length (cm)",
10     y = "Petal width (cm)"
11   )
```

PLOTTING IRIS



ggplot offers many plotting possibilities which we will go further into later. If you are eager, you can learn more [here](#) or [here](#).

SAVING PLOTS

You can save a plot by clicking on the **Export** button in the **Plots** window (bottom right window by default).

Save your plots as **.svg** if your text editor supports it and if you are not limited by file sizes. Otherwise, save your plots as **.png**.

```
1 p |> ggsave("plot.png")
```



ANY

QUESTIONS

TO GO FURTHER

- Visit: [Coding Club](#)
- EDGE coding club: Every 14 days, we have a coding club on Thursdays at 1:00 pm in the 5405 Rosshavet room (Natrium)

WHAT IS NEXT

- 16th of April 13:15am -17m *2122 lapetus ALC*
- Practice using R (self-study)
 - Reviewing and practicing the material we covered in today's lecture
 - Linear models

